

TP Lab 3 — Analogue-Digital Converter (ADC)

PIC18F4525 | MPLAB X | Compilateur XC8

Modules communs à tous les exercices

Ces deux fonctions sont utilisées dans les 3 exercices. À copier dans chaque projet.

init_ADC() — Initialisation du module ADC

Registres à configurer :

- **ADCON1 = 0x0E** : PCFG=1110 → seul AN0 (RA0) est analogique, reste = numérique
- **ADCON0 = 0x01** : CHS=0000 (canal 0=AN0), ADON=1 (ADC activé)
- **ADCON2 = 0b10110111** : ADFM=1 (résultat à droite), ACQT=110 (16 TAD), ADCS=111 (FRC)

```
void init_ADC(void) {  
    ADCON1 = 0x0E;           // AN0 seul en analogique, reste = numerique  
    ADCON0 = 0x01;           // Canal AN0 (CHS=0000), ADC ON (ADON=1)  
    ADCON2 = 0b10110111;     // ADFM=1 (droite), ACQT=16TAD, ADCS=FRC  
}
```

Décodage ADCON2 = 0b10110111 : bit 7 ADFM = 1 → résultat justifié à DROITE (valeur 0-1023 dans ADRESH:ADRESL) bit 6 = 0 → non utilisé bits 5:3 ACQT= 110 → temps d'acquisition = 16 TAD bits 2:0 ADCS= 111 → horloge = FRC (oscillateur RC dédié à l'ADC)

conversion() — Démarrer et lire la conversion ADC

- Mettre GO_nDONE à 1 → déclenche la conversion
- Attendre que GO_nDONE revienne à 0 (conversion terminée automatiquement)
- Résultat : ADRESH (2 bits hauts) + ADRESL (8 bits bas) = valeur 10 bits (0 à 1023)

```
unsigned int conversion(void) {  
    ADCON0bits.GO_nDONE = 1;           // Demarrer la conversion  
    while (ADCON0bits.GO_nDONE);       // Attendre la fin  
    return ((unsigned int)ADRESH << 8) | ADRESL; // Retourner valeur 10 bits  
}
```

Pourquoi 'unsigned int' et pas 'int' ? L'ADC retourne une valeur de 0 à 1023. Un 'int' signé irait de -32768 à +32767, mais par convention le TP impose 'unsigned int' pour indiquer que la valeur est toujours positive (jamais négative).

Exercice 1 — Chirping Buzzer (Buzzer à fréquence variable)

Objectif : Utiliser l'ADC pour faire varier la fréquence du buzzer.

- Potentiomètre sur **RA0** (AN0) — Buzzer sur **RC2**
- Tourner le potentiomètre → change la fréquence du son (effet 'chirping')

buzzer_on(int delay) — Son à fréquence variable

Pourquoi ne pas utiliser `__delay_ms(val)` ?

`__delay_ms()` exige une **constante** connue à la compilation. On ne peut pas passer une variable.

→ On utilise une **boucle for** à la place : le nombre d'itérations = le délai.

```
void buzzer_on(int delay) {
    int i;
    LATCbits.LATC2 = 1;           // RC2 = 1 (buzzer ON)
    for (i = 0; i < delay; i++);   // Attendre 'delay' iterations
    LATCbits.LATC2 = 0;           // RC2 = 0 (buzzer OFF)
    for (i = 0; i < delay; i++);   // Attendre 'delay' iterations
    // -> Plus delay est grand -> periode plus longue -> son grave
    // -> Plus delay est petit -> periode plus courte -> son aigu
}
```

Code complet — Lab3_Ex1_Chirping_buzzer.c

```
#include <xc.h>
#pragma config FOSC = INTIO67, WDT = OFF, LVP = OFF, PBADEN = OFF
#define _XTAL_FREQ 8000000

// -- init_ADC() et conversion() (voir modules communs ci-dessus) --

void buzzer_on(int delay) {
    int i;
    LATCbits.LATC2 = 1;
    for (i = 0; i < delay; i++);
    LATCbits.LATC2 = 0;
    for (i = 0; i < delay; i++);
}

unsigned int val;    // IMPORTANT : unsigned int, pas int !

void main(void) {
    OSCCON = 0x72;    // Oscillateur interne 8 MHz
    TRISAbits.TRISA0 = 1;    // RA0 = entree (potentiometre AN0)
    TRISCbits.TRISC2 = 0;    // RC2 = sortie (buzzer)
    LATCbits.LATC2 = 0;    // Buzzer eteint au demarrage

    init_ADC();          // Initialiser l'ADC

    while (1) {
        val = conversion();    // Lire le potentiometre (0 a 1023)
        buzzer_on(val);        // Produire le son avec ce delai
        // Tourner le potentiometre -> frequence du son change
    }
}
```

Comment ça marche : Potentiomètre à 0 → ADC = 0 → delay = 0 → son très aigu (très rapide) Potentiomètre à 50% → ADC = 512 → delay = 512 → son moyen Potentiomètre à max → ADC = 1023 → delay = 1023 → son très grave (très lent)

Exercice 2 — ADC LED (Affichage sur LEDs)

Objectif : Allumer les LEDs en fonction de la valeur du potentiomètre.

- Potentiomètre sur **RA0** (AN0) — LEDs sur **RB0 à RB3** (PORTB)
- ADC 10 bits → plage 0-1023 divisée en 4 zones de 256

Tableau de mapping ADC → LEDs

Valeur ADC	Tension approx.	LATB (binaire)	LEDs allumées
0 à 255	0 à ~1.25 V	0b00000001	1 LED (RB0)
256 à 511	~1.25 à ~2.5 V	0b00000011	2 LEDs (RB0-RB1)
512 à 767	~2.5 à ~3.75 V	0b00000111	3 LEDs (RB0-RB2)
768 à 1023	~3.75 à 5 V	0b00001111	4 LEDs (RB0-RB3)

Code complet — Lab3_Ex2_ADC_LED.c

```
#include <xc.h>
#pragma config FOSC = INTIO67, WDT = OFF, LVP = OFF, PBAEN = OFF
#define _XTAL_FREQ 8000000

// -- init_ADC() et conversion() (voir modules communs) --

unsigned int val;

void main(void) {
    OSCCON = 0x72;
    TRISB = 0x00;           // PORTB en SORTIE (LEDs sur RB0-RB3)
    TRISAbits.TRISA0 = 1;   // RA0 = entree (potentiometre AN0)
    LATB = 0x00;           // LEDs eteintes

    init_ADC();

    while (1) {
        val = conversion(); // Lire ADC (0 a 1023)

        if (val < 256) {
            LATB = 0b00000001; // 1 LED allumee (~0 a 1.25V)
        } else if (val < 512) {
            LATB = 0b00000011; // 2 LEDs allumees (~1.25 a 2.5V)
        } else if (val < 768) {
            LATB = 0b00000111; // 3 LEDs allumees (~2.5 a 3.75V)
        } else {
            LATB = 0b00001111; // 4 LEDs allumees (~3.75 a 5V)
        }
        // Tourner le potentiometre -> les LEDs s'allument progressivement
    }
}
```

Explication de LATB vs PORTB : On utilise LATB (pas PORTB) pour écrire sur le port. PORTB lit l'état physique des broches, LATB écrit la valeur sur les sorties. Toujours utiliser LAT pour écrire, PORT pour lire.

Exercice 3 — Voltmètre avec Afficheur LCD

Objectif : Afficher la tension du potentiomètre sur un LCD.

- Potentiomètre sur **RA0** (AN0)
- Afficheur LCD : données sur **PORTD**, contrôle sur **PORTE**
- Ligne 1 : titre fixe — Ligne 2 : tension en temps réel

Étapes pour ajouter la bibliothèque LCD dans MPLAB X

1. Télécharger **LCD_Library.c** et **LCD_Library.h** depuis Moodle
2. Copier-coller les 2 fichiers dans le dossier du projet
3. Dans MPLAB X : clic droit **Source Files** → **Add Existing Item** → choisir LCD_Library.c
4. Dans **Header Files** : Add Existing Item → choisir LCD_Library.h
5. Ajouter **#include "LCD_Library.h"** dans votre fichier .c

Formule de conversion ADC → Tension

ADC 10 bits : **0 à 1023** correspond à **0 à 5V** (VDD = 5V)

Formule : **$V = val \times 5.0 / 1023.0$**

Exemples : val=0 → 0.00V | val=512 → 2.50V | val=1023 → 5.00V

Déclarer V en float (pas int !) car la tension a des décimales.

Code complet — Lab3_Ex3_Voltmeter.c

```
#include <xc.h>
#include <stdio.h>          // Pour sprintf()
#include "LCD_Library.h"    // Telecharger sur Moodle !
#pragma config FOSC = INTIO67, WDT = OFF, LVP = OFF, PBADEN = OFF
#define _XTAL_FREQ 8000000

// -- init_ADC() et conversion() (voir modules communs) --

unsigned int val;    // Valeur brute ADC (0 a 1023)

void main(void) {
    char  buffer[17];    // Tampon pour sprintf (16 chars + fin de chaine)
    float v;             // Tension calculee (FLOAT, pas int !)

    OSCCON = 0x72;       // Oscillateur 8 MHz
    TRISD  = 0x00;       // PORTD en SORTIE (donnees LCD)
    TRISE  = 0x00;       // PORTE en SORTIE (RS, RW, E du LCD)
    TRISAbits.TRISA0 = 1; // RA0 en entree (potentiometre AN0)

    init_ADC();           // Initialiser l'ADC

    lcd_init();           // Configurer le LCD
    lcd_on();             // Allumer l'affichage

    goto_line(1);         // Curseur sur ligne 1
    print_string("PIC18F4525"); // Titre fixe

    while (1) {
        goto_line(2);     // Curseur sur ligne 2
        val = conversion(); // Lire ADC (0-1023)
        v = (float)val * 5.0 / 1023.0; // Convertir en tension
        sprintf(buffer, "Vol: %4.2f V ", v); // Formater (ex: "Vol: 3.25 V")
        print_string(buffer); // Afficher sur LCD
        // Tourner le potentiometre : tension varie de 0.00 a 5.00 V
    }
}
```

Explication de sprintf : sprintf(buffer, "Vol: %4.2f V", v) %4.2f → affiche un float avec 4 chiffres au total et 2 décimales Exemple : v = 3.252 → affiche '3.25' Les 2 espaces à la fin (' ') effacent les anciens caractères sur le LCD.

Fonctions LCD à connaître

Fonction	Ce qu'elle fait
lcd_init()	Configure et initialise le LCD
lcd_on()	Allume l'affichage
lcd_clr()	Efface tout le LCD
goto_line(n)	Place le curseur au début de la ligne n (1 ou 2)
print_string(s)	Affiche la chaîne de caractères s sur le LCD

Récapitulatif — Points clés Lab 3

1. ADCON1 = 0x0E → seul AN0 est analogique (RA0 = potentiomètre).
2. ADCON2 = 0b10110111 → résultat à **droite** (0-1023), 16 TAD, FRC.
3. Déclarer les variables ADC en **unsigned int** (jamais int).
4. Pour un délai variable : utiliser une **boucle for**, pas `__delay_ms()`.
5. Pour la tension : déclarer en **float** et utiliser la formule $V = \text{val} \times 5.0 / 1023.0$.
6. Toujours utiliser **LAT** pour écrire sur un port (pas PORT).

Schéma de connexion résumé

```
PIC18F4525
RA0 ---[Potentiometre : milieu]--- RA0      (AN0 = entree ADC)
      [une extremite]             --- GND
      [autre extremite]          --- +5V
RC2 ---[Buzzer piezoelectrique]--- GND      (Ex 1)
RB0-RB3 ---[LEDs avec R 470Ω]--- GND        (Ex 2)
PORTD (RD0-RD7) --- [Donnees LCD D0-D7]     (Ex 3)
RE0 (RS), RE1 (RW), RE2 (E) --- [Controle LCD] (Ex 3)
```